

C - -

INFORME STAGE III

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.3

Nombre	Apellido	Legajo	E-mail
Agustin	Galan	64098	aggalan@itba.edu.ar
Nicolas	Koron	64094	nkoron@itba.edu.ar
Nicolas	Bellavitis Alzate	64001	nbellavitisalzate@itba.edu.ar

1. Introducción	2
2. Modelo Computacional	3
2.1. Dominio	3
2.2. Lenguaje	3
3. Implementación	4
3.1. FrontEnd	4
3.2. BackEnd	5
4. Futuras Extensiones	8
5. Conclusiones	8

1. Introducción

El presente trabajo práctico consiste en el desarrollo de un compilador para un lenguaje imperativo simplificado, denominado C--, cuyo objetivo es traducir programas escritos en dicho lenguaje a código en C. La iniciativa surge a partir del análisis de C como lenguaje para aprender a programar, ya que si bien es poderoso y ampliamente utilizado, presenta complejidades sintácticas y semánticas que pueden dificultar su aprendizaje, especialmente para principiantes.

Ante esta situación, planteamos la creación de un lenguaje más accesible que conserve la estructura general de C pero que, al mismo tiempo, elimine los elementos que suelen generar confusión. El resultado es C--, un subconjunto más claro, expresivo y legible, pensado como herramienta educativa para introducir conceptos fundamentales de la programación imperativa sin exponer al estudiante a los aspectos más técnicos del lenguaje C original.

En este informe se detalla el diseño de C--, las decisiones tomadas durante su implementación, y el funcionamiento general del compilador, desde el análisis léxico y sintáctico hasta la generación de código C equivalente.

2. Modelo Computacional

2.1. Dominio

El dominio del lenguaje C-- está basado en el paradigma de programación imperativo, heredado directamente del lenguaje C. El objetivo principal de este compilador es permitir la escritura de programas simples, estructurados, y con un comportamiento predecible, ideal para tareas académicas, algoritmos básicos, y la enseñanza de estructuras de control.

En este lenguaje las variables, tanto escalares como arreglos, se representan mediante identificadores y son almacenadas en sus respectivos scopes. Cada función genera un nuevo ámbito local, y la ejecución se realiza de forma secuencial, instrucción por instrucción.

A nivel de tipos de datos, C-- trabaja con una tipificación estática y fuerte, y admite los siguientes tipos básicos:

- **INT**: enteros con operaciones aritméticas básicas.
- **BOOL**: booleanos, con expresiones lógicas y condicionales.
- **STRING**: cadenas de texto.
- Arreglos de los tipos anteriores: **INT[]**, **BOOL[]**, **STRING[]**.

El modelo computacional incluye funciones, estructuras de control (**IF**, **WHILE**, **FOR**, **MATCH**), declaraciones de variables, llamadas a funciones, uso de macros simples (con parámetros formales), e impresión de valores en consola.

2.2. Lenguaje

El lenguaje C-- fue diseñado como un subconjunto simplificado y estructurado de C. Entre las características más destacadas del lenguaje se encuentran:

- **Funciones**: definidas con tipos explícitos (**INT**, **BOOL**, **VOID**, **STRING**) y listas de argumentos. La declaración inicia un nuevo ámbito léxico.
- **Declaraciones**: permiten inicializar variables con o sin valores al momento de la declaración.
- **Control de flujo**: incluye **IF** con **ELSE**, **WHILE**, **FOR**, y **MATCH**, este último similar a un **switch** pero extendido para trabajar tanto con enteros como con cadenas.

- **Operaciones aritméticas y lógicas:** soporta operaciones básicas (+, -, *, /, ==, !=, <, <=, >, >=, &&, ||, !, ++, --).
- **Arreglos:** unidimensionales, con acceso por índice y asignaciones por posición.
- **Macros:** definidos con el keyword **MACRO**, permiten crear funciones simples reutilizables con parámetros formales y un cuerpo que representa una expresión matemática.
- **Funciones recursivas:** las funciones pueden llamarse a sí mismas.

3. Implementación

3.1. FrontEnd

El desarrollo frontend se centró en definir una gramática que sea similar a C pero que elimine las complejidades del mismo. Para una primera instancia se utilizó Flex, encargado de transformar un programa de entrada en el lenguaje desarrollado en un stream de tokens. En el archivo FlexPatterns.l se definió un alfabeto, y se utilizaron los contextos de Flex para tratar lexemas de una forma distinta, por ejemplo, en comentarios y cadenas de caracteres (Strings).

Cabe destacar que en nuestro caso en particular, fue necesario utilizar una tabla de símbolos desde Flex para poder reconocer cuando un identificador ya había sido previamente declarado, para emitir el token adecuado. Esto se debe a que nuestra gramática distingue entre identificador de enteros, booleanos, funciones, etc.

Utilizamos Flex y la estructura de lexicalAnalyzerContext para asignarle un valor semántico a los lexemas que Flex reconocía. Por ejemplo, al token INTEGER se le asociaba como valor semántico el número recibido como entrada. Con ciertos tokens como los operadores de comparación se optó por asignar el propio token como valor semántico. Todos los tokens son utilizados por Bison para la etapa de análisis sintáctico.

Para realizar dicho análisis, en Bison, se definen tanto los símbolos terminales como no-terminales para posteriormente utilizarlos con reglas de producción. Ejemplos de símbolos terminales definidos son 'INT', 'SORT', 'RETURN', 'VOID' 'MACRO'.

Como estructura general de la gramática se puede destacar lo que llamamos declaraciones externas, que podían ser tanto declaraciones de funciones y macros, o sentencias. El programa está compuesto por una lista de declaraciones externas. Por otro lado, las sentencias que definimos son, entre otras cosas, las operaciones sobre variables y control de flujo propias del lenguaje imperativo. Se puede destacar que tienen en común que representan acciones concretas. Si bien muchas sentencias pueden tener otras sentencias como parte de su cuerpo, una sentencia tiene una acción completa claramente definida. Asimismo, siempre terminan con un salto de línea o un bloque de sentencias, delimitado por llaves.

3.2. BackEnd

Para el BackEnd se creó una tabla de símbolos, así como también se utilizó un stack con scopes, el cual, en conjunto con la tabla, permiten realizar un uso de variables más amigable para el usuario. Es importante aclarar para hablar de esta etapa, que la definición de la gramática distinguiendo en todo momento entre expresiones de Strings, expresiones de booleanos, y expresiones matemáticas o de enteros, reduce significativamente las responsabilidades del backend. Esto se debe a que gran parte de los chequeos de tipos y semánticas válidas está integrado a la gramática.

La tabla de símbolos es utilizada principalmente por Flex, como fue mencionado anteriormente, para el chequeo del tipo de dato asociado a los identificadores, emitiendo tokens distintos para los identificadores de distintos tipos de datos. Para la tabla de símbolos, se definió la siguiente estructura:

```
typedef struct {
    char *name;           // Nombre del símbolo
    Type *types;          // Arreglo de tipos (En funciones, primer valor es el
                          // tipo de retorno)
    int typeCount;        // Cantidad de tipos
    Scope scope;          // Alcance del símbolo (local, global, etc.)
    SymbolType symbolType; // Función, variable, macro.
} Symbol;

typedef struct {
    Symbol *symbols[MAX_SYMBOLS];
    Symbol *currentFunction;
    int count;
} SymbolTable;
```

Se utiliza el arreglo para guardar todos los símbolos definidos, incluyendo también el Scope asociado dentro del struct Symbol. Se optó por guardar los tipos de datos de un símbolo como un array, donde las variables tendrán un array de largo uno, con el tipo de dato de la variable. Al mismo tiempo, las macros o funciones tienen un array de largo que puede ser mayor a uno, y el primer valor de dicho arreglo se asocia al tipo de retorno.

También se incluyó en cada símbolo un valor del enum SymbolType, que distingue si el símbolo es una variable, función o macro. Esto no se consideraba necesario inicialmente, ya que normalmente se puede distinguir según el largo del arreglo de símbolos si se trata de una variable o no. No obstante, era más complejo distinguir entre macros y funciones, por lo que se consideró que el uso del enum sería lo más sencillo y prolijo.

Por último, un símbolo también cuenta con un Scope asociado (que manejamos como enteros). Este Scope es asignado según la lógica del stack de scopes, cuya estructura es la siguiente:

```
typedef struct {
    Scope * scopes;
    int size;
    Scope * lastScope;
} ScopeStack;
```

Donde cada scope es un entero.

Se incluyen los scopes activos en el stack de scopes, y se quitan a medida que dejan de estar en uso. La variable lastScope se utiliza para incrementar secuencialmente el valor de los scopes. La variable size simplemente indica el índice del último valor del stack. A la hora de buscar un

símbolo, se analiza que su scope esté dentro del stack de scopes activos, ya que si no se encuentra en el mismo, se considera que fue una variable local que ya no está declarada. Asimismo, las macros y funciones tienen asignadas el scope 0, el cual se considera el scope global, ya que es el más bajo de todos y siempre está incluido en el stack.

Para el chequeo de los tipos de datos que se retornan dentro de las funciones se utilizó una variable "currentFunction" ubicada dentro del struct de la tabla de símbolos

```
typedef struct {  
    Symbol *symbols[MAX_SYMBOLS];  
    Symbol * currentFunction;  
    int count;  
} SymbolTable;
```

La cual se actualiza cada vez que se entra a una definición de función, luego en cada construcción de returnStatement se obtiene el símbolo de la función y se revisa que coincida con el tipo de la última función guardada. Para la validación de los parámetros de las funciones, tanto el largo como el tipo de dato cada vez que se hace una invocación de función en la construcción del nodo del árbol se busca el símbolo correspondiente al nombre de la función y se hace un chequeo de tipo de dato uno a uno con los parámetros guardados en la tabla de símbolos y con los que se hizo la llamada de función, si alguno no coincide el tipo de dato o la cantidad es diferente se marca con un error.

Para la generación de código se agregó un prólogo donde siempre en la salida se incluyen librerías, una función de ordenamiento de arrays enteros para que nuestra función SORT tenga funcionalidad y otra para poder traducir el MATCH de string en c ya que el switch no acepta strings, también para el código que no se encuentre dentro de una función se agregó un wrapper de main ya que código suelto no compilaría dentro de c, por lo que imprime primero todas las definiciones de función, luego las declaraciones de variables y arreglos, y por último el resto del código.

En caso de que ya exista una función main, dado que main no significa nada en nuestro código y la idea es que ejecute el código que se encuentra fuera de funciones predefinidas primero en el código de salida se le cambia el nombre a la función main del usuario por main2, afectando también sus llamados.

3.3. Dificultades Encontradas

Entre las dificultades más significativas se encuentran:

Conflictos de tipo SHIFT-REDUCE y REDUCE-REDUCE en la definición de las producciones de las boolean expressions como las math expressions. El problema se originó cuando el equipo decidió incluir en algunas cláusulas (por ejemplo IF, WHILE) exclusivamente expresiones booleanas. Ejemplos de esto sería:

IF (1 + 1 > 3 OR flag)

WHILE (TRUE)

IF (1 + 1 = 2)

El problema residía en que tanto las expresiones booleanas como matemáticas tenían como producciones un identificador, por lo que había dos posibles árboles que se podían generar, creando conflictos. Al encontrarnos con ese problema, originalmente pensamos que no tenía solución, y optamos por unificar las expresiones matemáticas y booleanas, eliminando el conflicto pero perdiendo la capacidad de hacer la gramática más restrictiva.

Al mismo tiempo, acudimos a los docentes para encontrar la mejor solución posible. Luego de una larga discusión, se decidió por reestructurar la gramática, volviendo a la idea de utilizar expresiones booleanas y matemáticas por separado, pero distinguiendo los identificadores que se utilizan según su tipo de dato. De esta forma, eliminamos los conflictos REDUCE/REDUCE que traían trabajar solo con un tipo de identificador, y logramos mantener los chequeos de expresiones válidas dentro de la gramática sin tener que delegar a etapas posteriores. Cabe destacar que para que esto funcione, fue necesario incluir la tabla de símbolos.

Otra dificultad que encontramos fue que al querer redefinir una variable en un scope distinto al original de la definición de esa misma variable, Flex en vez de retornar un GENERIC_ID como debería devuelve un STRING_ID o INT_ID según corresponda, ya que encontraba una entrada anterior para ese identificador en la tabla de símbolos. Esto hacía que falle la compilación del programa a pesar de que este sea válido.

Para solucionar esto agregamos un flag en el CompilerState llamado DeclarationMode, cuya funcionalidad es que cuando en Bison comienza alguna de las producciones asociadas a declaración de variables, funciones o macros, acciona una mid-rule action. Esta acción hace que el flag de declaración sea verdadero. Al mismo booleano se le asigna el valor de falso cada vez que se emite un token de identificador.

Eso, combinado con las funciones de buscar símbolos en la tabla de símbolos, permite que si el flag es verdadero y el símbolo está definido en un scope distinto o no está definido devuelve un GENERIC_ID y si está definido en el scope actual devuelve el tipo de dato del símbolo.

De esta forma, se logró hacer posible la redefinición de una variable en scopes anidados.

En las funciones recursivas nos encontramos con un problema ya que estas se guardaban en la tabla de símbolos una vez que estas fueron completamente declaradas. Es decir, las funciones se agregan a la tabla de símbolos una vez que la regla de producción era encontrada en su totalidad. Por lo tanto, cuando se procesaba la regla de producción del return en Bison, la función aún no había sido agregada a la tabla de símbolos. Esto generaba inconvenientes en el siguiente ejemplo:

```
INT funcion_recursiva (INT a) {  
    ...  
    RETURN funcion_recursiva( 3 )  
}
```

Los problemas se debían a que a la hora de hacer un llamado a función, como “funcion_recursiva(3)” en este ejemplo, el identificador funcion_recursiva no había sido agregado a la tabla, por lo tanto Flex lo reconocía como un identificador genérico. Esto hacía que el análisis sintáctico falle porque no hay reglas para llamados a función con identificadores genéricos.

Entonces, la solución fue que la función sea agregada a la tabla de símbolos inmediatamente después de que sean declarados los parámetros (es decir, después del ‘)’) y no después de que se reconozca la totalidad del cuerpo de la función, haciendo que cuando se procese la línea de RETURN la función y su tipo de retorno ya se encuentren en la tabla de símbolos. Esto se hizo utilizando nuevamente mid-rule actions de Bison.

4. Futuras Extensiones

Si bien el lenguaje C - - fue concebido como una herramienta educativa con un conjunto reducido de características, existen varias direcciones en las que podría dirigirse para ganar mayor expresividad sin perder simplicidad. Una de las primeras extensiones posibles sería la incorporación de tipos compuestos, como estructuras (**STRUCTS**), lo cual permitiría representar datos más complejos y fomentar buenas prácticas de organización del código desde etapas tempranas del aprendizaje.

Otra mejora potencial sería el soporte para funciones anónimas o expresiones lambda, lo que habilitaría un estilo más flexible y modular, especialmente en funciones que involucren ordenamientos, filtros o recorridos. Asimismo, se podría considerar la inclusión de tipos genéricos, útiles para definir funciones y estructuras de datos reutilizables, manteniendo la seguridad de tipos en tiempo de compilación.

Finalmente, también podría incorporarse una biblioteca estándar más extensa, con funciones predefinidas para operaciones sobre cadenas, listas, entrada/salida, y manejo básico de errores. Esto permitiría a los estudiantes enfocarse más en la lógica de resolución de problemas sin necesidad de implementar desde cero funcionalidades auxiliares comunes, que suele generar mucho estrés y espantar a los estudiantes, comparado con Java por ejemplo.

5. Conclusiones

El desarrollo de este compilador para el lenguaje C- - nos permitió explorar en profundidad los distintos componentes que conforman un lenguaje de programación, desde el diseño léxico y sintáctico hasta la implementación de acciones semánticas y la generación de código intermedio en C. A través de este proceso, buscamos construir un lenguaje imperativo simplificado, pensado especialmente para facilitar el aprendizaje de quienes se inician en la programación.

Esto nos llevó a comprender mejor las dificultades que se enfrentan a la hora de diseñar un lenguaje, y también nos desafió a buscar un balance entre simplicidad y poder de expresión, entendiendo que ambas son importantes.

C- - logra mantener una estructura simple e intuitiva , permitiendo trabajar con variables, funciones, estructuras de control, arreglos y macros, todo en un entorno más amigable y menos propenso a errores comunes de sintaxis. Si bien se trata de un lenguaje reducido, su diseño permite futuras extensiones sin comprometer su simplicidad.

En definitiva, este trabajo no solo nos permitió aplicar conceptos fundamentales de teoría de lenguajes y compiladores, sino también reflexionar sobre cómo el diseño de un lenguaje influye en la experiencia del programador.